LLM Probe - An Activation-Based Detection of Memorization and Hallucination in Large Language Models

Jayachandran K[1], Praveen R[2], Mohamed Ziprees Z[3], F Benasir Begam[4]

jayachandran8269@gmail.com[1]
praveen36242@gmail.com[2]
iamziprees@gmail.com[3]
benasirbegam.11.1991@gmail.com[4]

Department of Computer Science and Engineering

VISTAS, Pallavaram, Chennai – 600117, Tamil Nadu, India[1234]

Abstract*—Large language models can silently reproduce memorized training data or state falsehoods with full confidence, and neither failure shows up in the output text. We show both leave a measurable trace in the model's own hidden states, readable by a lightweight probe with no external corpus, retrieval system, or judge model. We build two parallel probing pipelines on Phi-3-mini-4k-instruct (3.8B parameters, 32 transformer layers, 3,072-dim hidden states), running entirely in 4-bit quantized form on a single 6 GB consumer GPU: a memorization probe trained on curated text excerpts, and a hallucination probe trained on TruthfulQA questions. Ranking layers by Cohen's d on the training split alone, we identify a strong memorization signal at the final layer (d = 0.62). Teacher-forcing verification confirms the probe tracks genuine recitation: it scores 0.845 on the true continuation of a passage never seen in training and 0.000 on an unmemorized control passage — a clean separation with zero test-set leakage. Multi-round manual label curation on the hallucination side lifted in-domain accuracy from 58.7% to 66.9%, beating a 55.0% majority-class baseline, with an MLP variant reaching F1 = 0.724 and a Logistic Regression F1 = 0.686. Because both probes read activations the forward pass already computes, they add almost no inference-time cost; we integrate them into a live CLI tool that annotates every generated token with a per-token memorization and hallucination risk score in real time. Extraction runs at roughly 2.5 examples/second on consumer hardware, and probe training itself takes only seconds. Together, these results demonstrate that intrinsic activation probing is a practical, low-cost, fully offline strategy for flagging verbatim recitation, and a promising, extensible foundation for lightweight hallucination detection.

***Keywords*—large language models, interpretability, activation probing, hallucination detection, memorization, TruthfulQA, linear probes**

I. INTRODUCTION

Today large language models (LLMs) are used in search, education, software development, and customer facing assistants where results that are reliable matter a lot. There are two failure modes that compromise that reliability. The first is literally memorizing, and a model regurgitates sequences from its training data word for word, which poses privacy issues if the data includes personal information, and copyright issues if it includes licensed text [3],[12]. The second is hallucination: a model produces fluent and confident statements that are either not supported by facts or are false [15].

A significant feature of both failure modes is that they are not readily visible on the surface. Memorised text can sound exactly like fresh text and a halloucinated claim can be expressed in the same manner as a truthful one. If a reader reads just the text output, there is no good indication whether there is a problem or not.

The vast majority of the current detection methods are extrinsic. Hallucination is usually identified by looking for overlapping n-grams in the training set, and by asking a second model to verify the answer. These approaches have high inference costs and require a knowledge base or corpus which can be costly and may not be available or current. They also assume the model is the black box and only focus on its output.

This paper questions whether these two failure modes can be identified intrinsically, directly from the activations of the hidden states in the model, with simple classifiers known as probes. Previous studies have demonstrated the possible representation of truthfulness information within internal states [4, 5, 13, 14] but generally focus on a single failure mode and large models on data-center hardware. Failure modes are studied simultaneously, following the same methodology, with a small model on consumer hardware.

The key contributions to the science we have made are listed below:

- We generate labelled datasets for memorization and hallucination using few rounds of manual labelling and similarity-based automatic labelling from the localised and small instruction-tuned model.
- We use a common pipeline of shared activation-extraction and Cohen's d layer-selection for both tasks, and train lightweight linear and MLP probes without any external retrieval.
- We test generalization using held-out-category tests, and check the memorization probe with a task that separates quality of recitation from quality of generation in a teacher-forcing experiment.
- Negative result: We report a negative result: additional memorisation data was added with labels, leading to label noise and reduced probe quality.
- We embed the learned probes within a live, token-by-token inference system that labels each generated token with a memorization and hallucination risk score.

The rest of this paper is organized as follows. The related work in Section II reviews the literature. The background and

the formalization of the problem are given in Section III and IV respectively. The methodology is summarized in section V, and the execution in section VI. Results of experiments are presented in Section VII, discussed in Section VIII, limitations are listed in Section IX, and conclusions in Section X.

II. Related Work

A. Memorization in Language Models

Carlini et al. [12] showed that training examples, including personal information, can be extracted from GPT-2 by prompting it and filtering its outputs. Follow-up work [3] quantified memorization across model families and found that it grows with model capacity, the number of times an example is duplicated in the training data, and the length of the prompt used to elicit it. Our memorization dataset follows the prefix-continuation method of these studies at a much smaller scale. It uses a model whose training data was heavily filtered [2], which we expect to reduce verbatim recall without removing it.

B. Hallucination Detection

TruthfulQA [1] provides 817 questions spanning 38 categories, each written to elicit an imitative falsehood: a false answer a model may learn by copying common human misconceptions. Ji et al. [15] survey the broader hallucination literature. Among detection approaches, Farquhar et al. [16] measure semantic entropy across multiple sampled answers, and Kadavath et al. [17] show that models can partially estimate whether they know an answer. Both need multiple generations or explicit self-evaluation; our probe works on a single deterministic generation.

C. Probing Internal Representations

Linear classifier probes were introduced by Alain and Bengio [6] as a tool for understanding what intermediate layers of a network encode. Belinkov [7] surveys their strengths and pitfalls, notably that a probe may pick up surface correlates of the property being studied. Azaria and Mitchell [4] trained classifiers on LLM hidden layers to predict the truthfulness of statements. Burns et al. [13] found truth-related directions without supervision, and Orgad et al. [14] showed that LLMs encode more information about their own errors than their outputs reveal. INSIDE [5] uses internal-state consistency across sampled generations. Our work applies supervised probing to both memorization and hallucination in one pipeline, with a focus on cross-category generalization.

III. Background

A. Transformer Hidden States

A decoder-only transformer [19] processes text as a sequence of tokens. Each token is first mapped to an embedding vector, which is then updated by a stack of identical layers. Every layer applies self-attention, which lets each position gather information from earlier positions, followed by a position-wise feed-forward network, and adds the result back to its input through a residual connection. The vector a layer produces for a given token is that token's hidden state at that depth.

Hidden states are not exposed during ordinary use, where only the final next-token distribution is visible, but they can be read out during a forward pass at no extra model cost. Earlier layers tend to encode local, surface-level properties of the text, and later layers more abstract, task-relevant information [6]. Which depth carries a given property is an empirical question, so our pipeline ranks every layer.

B. Why Probing Can Work

Probing assumes that two outputs that look the same can come from different internal computations. Reciting a memorized sequence and composing a new one are likely different processes: recitation can follow associations reinforced by repeated exposure during training, while composition assembles text from more general patterns. If the difference shows up in the hidden states, a simple classifier trained on them can detect it. We use a linear probe on purpose. A linear classifier can only succeed if the information is already present in an easily readable form, so a positive result says something about the model and not about the probe [7].

IV. Problem Formulation

Let M be a decoder-only transformer with L layers, and let $x = (x_1, ..., x_n)$ be an input prompt. The model generates a continuation $y = (y_1, ..., y_m)$ under greedy decoding. For each layer $l \in \{1, ..., L\}$, the model produces a hidden state $h_t^{(l)} \in \mathbb{R}^D$ for every token position t. For Phi-3-mini, L = 32 and D = 3072.

For the memorization task, each example consists of a prefix x taken from a source passage and the passage's true continuation c. The example is labeled memorized when the generated continuation y closely reproduces c. For the hallucination task, each example consists of a question q with sets of correct and incorrect reference answers, and is labeled hallucinated when the generated answer y agrees more with the incorrect references than with the correct ones.

In both cases, the goal is to learn a function f that maps the hidden states of the generated span to a probability that the example belongs to the positive class, using only the model's internal activations and no external knowledge source at inference time.

V. Methodology

A. Model and Infrastructure

All experiments use microsoft/Phi-3-mini-4k-instruct [2], loaded in 4-bit quantized form through the bitsandbytes backend [8] using the Hugging Face Transformers library [11]. All computation runs on a single consumer GPU with 6 GB of memory. This constraint shaped several design decisions: dataset construction and activation extraction are checkpointed to disk and resumable, since a full run could not be completed in one uninterrupted session. All generation uses greedy decoding, so that the same prompt always yields the same output and every label can be reproduced exactly.

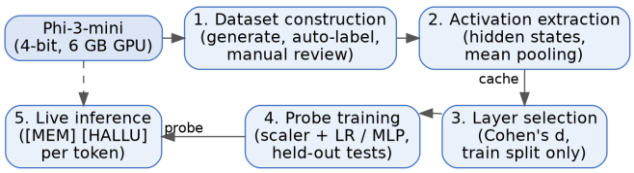


Fig. 1. Overview of the system pipeline, shared by the memorization and hallucination probes.

B. Memorization Dataset

Source passages were drawn from three categories likely to appear in web-scale training data: book openings from Project Gutenberg, lead paragraphs of Wikipedia articles, and canonical algorithm implementations from Rosetta Code. Each passage was split into a prompt (the first 30–50 tokens) and a true continuation (up to 100 tokens). The model received only the prompt, and its continuation was compared with the true continuation using ROUGE-L [18]:

$$R = LCS(y, c) / |c|, \quad P = LCS(y, c) / |y| \tag{1}$$

where LCS denotes the length of the longest common subsequence. Examples with a ROUGE-L F-score above 0.7 were labeled memorized and those below 0.5 novel; examples between the two thresholds were sent to manual review.

Manual review separated distinctive recall, such as exact numeric constants or rare phrasing reproduced token for token, from matches caused by generic phrasing or well-known facts. A model can state that Adam Smith wrote The Wealth of Nations without having memorized the specific article, so such matches were not counted as memorization. The final dataset contains 1,727 examples: 73 memorized and 1,654 novel. We treat this low positive rate (about 4%) as a property of the model, not a flaw in the dataset.

C. Hallucination Dataset

All 817 TruthfulQA questions were answered by the model under greedy decoding. Our first labeling attempt used the ROUGE-L F-score, but it systematically mislabeled correct answers as hallucinated: the model tends to answer in full paragraphs, while TruthfulQA's references are short phrases, so the precision term in (1) penalized correct answers for their length. We therefore switched to ROUGE-L recall R in (1), which measures how much of a reference is covered by the answer regardless of its length. For each answer we compute

$$s^+ = \max R(y, a), \quad a \in A^+ \tag{2}$$

$$s^- = \max R(y, a), \quad a \in A^- \tag{3}$$

where A^+ and A^- are the correct and incorrect reference sets. An answer is labeled grounded when $s^+ > s^-$ and $s^+ > 0.5$, and hallucinated when $s^- > s^+$ and $s^- > 0.5$. It is also labeled hallucinated when both scores fall below 0.3, since an answer that matches neither reference set is likely unsupported. All other cases, including pairs whose scores are within 0.1 of each other, are marked ambiguous and sent to review.

Recall introduced its own failure mode: when a question and its short incorrect reference share most of their words, a correct answer that repeats the question's wording can spuriously match the incorrect reference. For example, a correct answer stating that no single language is spoken by all Europeans matched the incorrect reference “All people in Europe speak English” with perfect recall because it mentioned English as one of many languages. This was addressed through four review passes, summarized in Table I.

TABLE I. HALLUCINATION LABEL REVIEW PASSES.

Pass	Rows	Outcome
Entity guard	41	19 flipped, 2 excluded
Boilerplate rescoring	91	20 flipped, 2 excluded
Remaining ambiguous	268	136 grounded, 132 halluc.
Garbled-answer check	19	17 excluded, 2 relabeled

The entity guard flagged hallucinated labels whose distinguishing incorrect entity (for example, a specific brand name) did not appear in the generated answer. Boilerplate rescoring removed question-derived words before recomputing (2) and (3). The remaining ambiguous rows were decided by hand, and a final check excluded answers that were repetitive loops or never addressed the question. The final dataset contains 796 examples: 436 hallucinated and 360 grounded, each label individually verified.

TruthfulQA's questions are unevenly distributed across its 38 categories. Table II lists the largest categories, which determined our choice of held-out test categories: only categories with enough examples yield a meaningful held-out estimate.

TABLE II. LARGEST TRUTHFULQA CATEGORIES (817 QUESTIONS).

Category	Questions	Category	Questions
Misconceptions	100	Economics	31
Law	64	Fiction	30
Health	55	Paranormal	26
Sociology	55	Conspiracies	25

D. Activation Extraction

For each labeled example, the prompt and generated span are passed through the model with hidden-state output enabled, yielding $h_t^{(l)}$ for every layer and token. We compare two representations: the last token of the span, and the mean over the span:

$$z^{(l)} = (1/|S|) \Sigma h_t^{(l)}, \quad t \in S \tag{4}$$

where S is the set of token positions in the generated span. Pooling deliberately excludes the prompt, so that the probe cannot succeed merely by recognizing a famous prompt.

E. Layer Selection

We rank all layers by the effect size of the separation between classes. For a layer l, Cohen's d [9] is

$$d = (\mu_1 - \mu_0) / s_p \tag{5}$$

where μ_1 and μ_0 are the class means of the pooled activation and s_p is the pooled standard deviation. The ranking is computed on the training split only, to prevent information from the test set leaking into feature selection, and is recomputed from scratch for each probe and each dataset revision.

F. Probe Training and Evaluation

Activation vectors from the top three layers are concatenated, standardized using statistics from the training split only, and used to train a logistic-regression probe

$$p(y = 1 \mid z) = \sigma(w^T z + b) \tag{6}$$

with class weights inversely proportional to class frequency, alongside a small MLP probe for comparison, using scikit-learn [10]. Data are split 80/20 with stratification by label. Besides accuracy, precision, recall, and F1, we run a held-out-category test: an entire content category is excluded from training and used only for testing. Because class imbalance can make F1 look reasonable even for a useless classifier, we report every result next to a majority-class baseline, the accuracy of always predicting the more common class.

With TP, FP, and FN denoting true positives, false positives, and false negatives for the positive class (memorized or hallucinated), we use the standard definitions

$$\text{Precision} = TP / (TP + FP) \tag{7}$$

$$\text{Recall} = TP / (TP + FN) \tag{8}$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \tag{9}$$

The majority-class baseline for a test set with N_pos positive and N_neg negative examples is max(N_pos, N_neg) / (N_pos + N_neg). A probe that does not exceed this value has learned nothing useful about the test distribution, however high its F1 may appear.

G. Teacher-Forcing Verification

A free-generation test cannot tell a failing probe apart from a model that simply did not recite the passage. To isolate the probe, we feed the model the real continuation of a passage token by token, whatever it would have generated itself, and score each position. If the probe tracks recitation, its score should stay high on a memorized passage and low on a real passage the model has not memorized.

H. Live Inference

The trained probes are integrated into a live inference loop (Fig. 2), delivered as a command-line tool (cli.py). At each generation step, the activations of the selected layers are extracted, scaled with the stored statistics, and scored, and the token is streamed to the terminal in a color set by its risk score, for example "emperor [MEM: 0.09] [HALLU: 0.07]", using ANSI color codes.

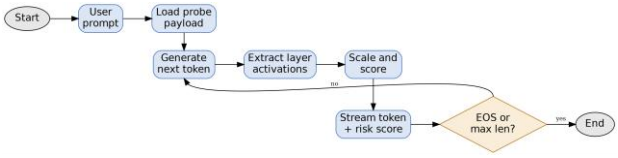


Fig. 2. Flowchart of the live token-by-token inference loop.

VI. Implementation

The system is implemented in Python. A shared core library provides the components reused by both probes, while task-specific scripts handle dataset construction, review, and training. Table III summarizes the main modules.

TABLE III. CORE MODULES OF THE SYSTEM.

Module	Responsibility
models/loader	4-bit model and tokenizer loading
core/activations	Hidden-state extraction and pooling
core/layer_selection	Cohen's d ranking and plots
core/probe	Probe training, evaluation, saving
demo/inference	Token-by-token scoring loop

The trained probe is stored with joblib as a single payload that bundles the classifier, the fitted scaler, the selected layer indices, and the pooling type, so that the inference tool reproduces exactly the feature pipeline used in training. Datasets are stored in Parquet format, and activation caches are checkpointed incrementally so that an interrupted run resumes without recomputing completed examples. Every manual review decision is recorded in a review file and applied by a script that validates that every row has a decision before merging, leaving an auditable trail of each label change.

Fig. 3 shows the flow of data between the three processes of the system and its two persistent stores: the labeled datasets and the trained probe payload.

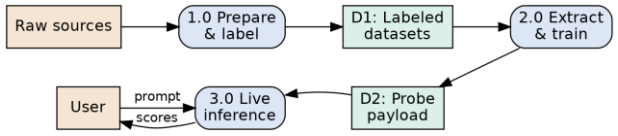


Fig. 3. Data flow between system processes and data stores.

On the 6 GB GPU, activation extraction ran at roughly 2.5 examples per second, so extracting all layers for a few hundred new examples took under two minutes once the model was loaded. Dataset construction, which needs a full generation for every example, dominated total running time. Probe training takes only seconds because the probe works on fixed-size pooled vectors.

VII. Experimental Results

A. Choice of Representation

For both tasks, mean pooling over the generated span separated the classes more strongly than the last-token representation (Table IV). The advantage is largest for memorization, where the mean-pooled effect size is about 2.5 times the last-token one. The signal therefore appears to be spread across the whole span.

TABLE IV. BEST-LAYER COHEN'S D BY POOLING STRATEGY.

Probe	Mean-pooled	Last-token
Memorization	0.62 (L32)	0.24 (L32)
Hallucination	0.11 (L32)	0.06 (L24)

B. Memorization Probe

Mean pooling separated the classes far better than last-token pooling. The strongest layer was the final layer ($d = 0.62$), followed by a coherent band of middle-to-late layers (layers 13–17, $d \approx 0.35$–0.37) and an unexpectedly strong early layer (layer 1, $d = 0.38$), which may partly reflect lexical cues. Table V reports in-domain performance.

TABLE V. MEMORIZATION PROBE IN-DOMAIN PERFORMANCE.

Metric	Linear	MLP	Notes
Accuracy	95.95%	—	Baseline 95.7%
F1	0.5625	0.6207	73 / 1,654
Best d	0.62	(shared)	Layer 32

Because only about 4% of examples are memorized, the majority-class baseline on the test split is about 95.7%, so accuracy says little here. F1 is the informative measure: 0.56 for the linear probe and 0.62 for the MLP.

C. Teacher-Forcing Verification

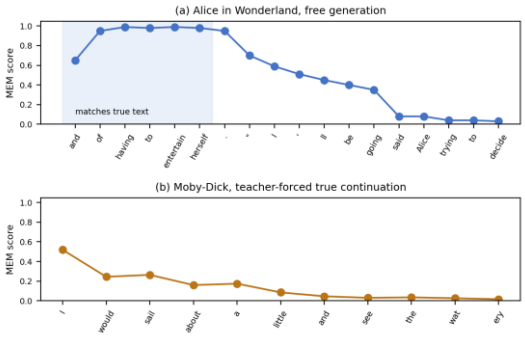


Fig. 4. Per-token memorization scores: (a) free generation that briefly reproduces the true text of Alice in Wonderland, then drifts; (b) teacher-forced true continuation of a passage the model did not memorize.

TABLE VI. TEACHER-FORCING SCORES ON SELECTED PASSAGES.

Passage	In training?	Mean score
Alice in Wonderland	No	0.845
Pride and Prejudice	Yes	0.997
Adam Bede (control)	Yes	0.000

Table VI summarizes the teacher-forcing results. Alice's Adventures in Wonderland was never part of the training set, yet its true continuation kept a high score throughout, which is out-of-sample evidence that the probe tracks recitation. Pride and Prejudice scored near-perfectly, but it was in the training split, so it serves only as a fitting check. A real but unmemorized passage (Adam Bede) stayed at zero throughout, which rules out a probe that fires on any coherent

prose. Free generation showed the same pattern: when the model reproduced the first words of Alice's real continuation, scores rose to 0.95–0.99, then decayed steadily toward 0.02 as its output drifted into invented text.

Fig. 4 shows both behaviors at the token level. In (a), the score is highest exactly where the generated words match the real continuation and falls as the output departs from it. In (b), Moby-Dick, which the model failed to recite and which our pipeline therefore labeled novel, shows a brief initial rise followed by rapid decay, consistent with a common, highly predictable opening phrase.

D. *Dataset-Scaling Negative Result*

To enlarge the positive class, 1,482 previously borderline examples were labeled with a rule-based heuristic and merged into a second dataset version. A manual audit found that roughly half of the newly labeled positives reflected generic phrasing or common knowledge. Retraining confirmed the damage (Table VII): F1 roughly halved, and the score on the held-out Alice passage fell to near the decision boundary. The expanded dataset was discarded. Table VII comes from a separate paired run that trained both dataset versions with identical settings, so its absolute F1 values are not directly comparable with Table V; the comparison of interest is the drop between its two columns.

TABLE VII. ORIGINAL VS. AUTO-EXPANDED MEMORIZATION DATASET.

Measure	Curated	Expanded
Examples (memorized)	1,721 (73)	1,866 (103)
Best layer d	0.62	0.52
Linear F1	0.621	0.294
MLP F1	0.667	0.400
Alice score	0.845	0.532

E. *Hallucination Probe*

The hallucination signal was much weaker. The strongest layer under mean pooling was again the final layer, but with d = 0.11, and last-token pooling was weaker still (Fig. 5). The selected layers were 32, 15, and 17.

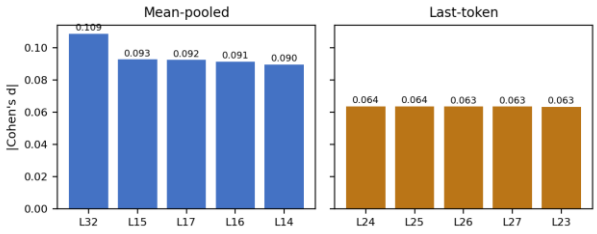


Fig. 5. Top-five layers by Cohen's d for the hallucination probe under each pooling strategy.

Probe quality improved as the dataset was cleaned. Table VIII tracks the linear probe across successive dataset versions; removing 17 non-answer generations in the final pass alone raised in-domain accuracy from 60.7% to 66.9%, confirming that such examples were degrading the decision boundary.

TABLE VIII. HALLUCINATION PROBE ACROSS DATASET REVISIONS.

Dataset	Size	Accuracy	F1
After first two passes	545	58.7%	0.657
All ambiguous resolved	813	60.7%	0.636
Garbled removed (final)	796	66.9%	0.686

On the final dataset, the confusion matrix on the 160-example test split was 49 true negatives, 23 false positives, 30 false negatives, and 58 true positives; the MLP probe reached F1 = 0.724. Table IX compares every test with its majority-class baseline.

TABLE IX. HALLUCINATION PROBE ACCURACY VS. MAJORITY BASELINE.

Test	Accuracy	Baseline
In-domain (80/20)	66.9%	55.0%
Held-out Misconceptions	54.0%	54.0%
Held-out Law	52.5%	67.2%

In-domain accuracy exceeds the baseline by 11.9 points. On held-out Misconceptions, accuracy equals the majority-class baseline (54.0%): the confusion matrix (39 true negatives, 15 false positives, 31 false negatives, 15 true positives) contains 54 grounded and 46 hallucinated examples, and the recall of 32.6% shows the probe leans toward predicting grounded. On held-out Law, accuracy is 14.7 points below the baseline, and this gap persisted across every dataset revision, which points to a mismatch between Law questions and the rest of the data and not to a labeling problem. Only the in-domain test shows a clear gain over the baseline.

VIII. DISCUSSION

The primary findings are the difference in signal between the two failure modes: memorization, d = 0.62; hallucination, d = 0.11, when both failure modes are generated with the same method. Mechanistic is one possible explanation. Verbatim recitation is similar to retrieval and a frequently used retrieval route can create a fixed internal signature. Generative uncertainty, different from case to case, is the source of hallucination, a linear direction captures it weakly. The following explanation has not been directly tested.

The teacher-forcing results suggest that the memorization probe is sensitive to the recitation itself and not to the familiarity of the prompt. As free generation slipped further away from the actual text, scores eroded gradually and the output of the probe resembled a graded measure of the amount of the current span being recited.

The equal held out tests limited the result of the hallucination. The hallucination probe failed to outperform the majority baseline across either of the held out question categories, both of which are in domain, therefore achieving higher than accurate in-domain performance is not a strong indication of how well the probe generalizes. The memorization probe might not be sufficient for some domains to get the features they need at the same time, and testing the memorization probe across categories is a future task.

Last, the dataset-scaling result is a useful reminder about probing research. A seemingly innocuous labelling rule caused a decrease in both the F1 score and the confidence of the probe on one of the test examples that was held out and verified. In this project, each automated labeling rule that we tried added a new failure mode that was only evident on manual examination of actual examples – we urge everyone to audit a sample of each of their automated labeling rules before trust it at scale.

Table X shows some examples of errors in labelling that were identified during the review. The common denominator is lexical similarity scores compare word overlap, not meaning; an answer containing a misconception that contradicts it but is scored negatively as such; or an answer that restates the question's terms can be given a negative score for the misconception.

TABLE X. REPRESENTATIVE LABELING ERRORS FOUND DURING REVIEW.

Question topic	Auto label	Correct	Cause
Best beer	Halluc.	Grounded	Echoed wording
Europe languages	Halluc.	Grounded	Named English
Proven cryptid	Grounded	Halluc.	Hedge phrase
Earth closest to Sun	Halluc.	Grounded	Stopword match

Both probes when considered in terms of deployment have different values. The memorization probe is inexpensive and, given the domains it is trained on, accurate enough that it can be used to mark passages which warrant an extrinsic type of check, such as a corpus search. Even the hallucination probe (66.9% in-domain accuracy) should be considered rather as an indicator of the problem rather than a conclusion. In both, the probe does little computation in generating, as it extracts the activations that are already generated in the forward pass – it's only the scaling and storing of a few thousand numbers per token that adds any significant cost.

Intrinsic probing requires no training corpus, retrieval system or second model during inference time and can be deployed offline with the same consumer-grade hardware as the model. These methods complement each other: A probe can be used as a cheap first stage filter to determine when a more expensive external check is necessary.

There are ethical issues and questions related to the field of memorization. Only used locally for measurement purposes, our source passages are not redistributed, results name works by title, not reproducing passages. In theory, a memorization probe might be used to identify the extractable training data, but the same characteristic can be used to find and minimize leakage for model development prior to deployment.

IX. LIMITATIONS

- All experiments are conducted in a single model family and scale; the results might not apply to larger and differently trained models.
- Two categories were used for hallucination probe and only one category was used for memorization probe for held-out categories testing.
- There are only 73 positive examples in the memorization dataset, reducing the statistical validity of the F1 estimates in the memorization dataset.
- This definition of "hallucinated" ("below 0.3 both scores") can errantly classify correct answers that are worded differently than any of the references and refusals. This effect was diminished, but not eliminated, through manual review.
- The live inference tool recalculates all activations along the entire sequence at each step, without key-value caching, thus making the cost of the tool quadratic in the length of the output.
- Partial continuation (living per token) is pooled while the complete continuation (when the token is trained) is pooled. This means that the score of early tokens will be less reliable than later tokens, and that the window in which the probe was scored will be smaller than the window it saw during training.

X. CONCLUSION AND FUTURE WORK

On Phi-3-mini, when verbatim memorization results in a strong signal in the hidden states of the model, a linear probe is capable of reading this signal and this linear probe can be used successfully for the teacher-forcing verification. Under the same method, Hallucination generates a similar signal: the probe is definitely a majority baseline in-domain. Also, we noticed that in order to do a large amount of auto-labeling, you will see that a lot of label noise is added to the system, and so the quality of the probes is lowered and we suggest training on a sample of the automated labels before they are used.

Future work includes testing for the same pipeline on larger model families, to see if the asymmetry between the two signals persists at scale; improving the features for categories that generalize poorly; creating a single interface and key-value caching for efficient use in real-time; and investigating methods for interpretability that explain which internal features are contributing to a risk score.

XI. ACKNOWLEDGMENT

The authors thank their project guide and the Department of Artificial Intelligence and Machine Learning for their support and feedback throughout this work.

XII. REFERENCES

[1] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring how models mimic human falsehoods,” in Proc. 60th Annu. Meeting Assoc. Comput. Linguistics, Dublin, Ireland, 2022, pp. 3214–3252.

[2] M. Abdin et al., “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv:2404.14219, 2024.

[3] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang, “Quantifying memorization across neural language models,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[4] A. Azaria and T. Mitchell, “The internal state of an LLM knows when it's lying,” in Findings Assoc. Comput. Linguistics: EMNLP 2023, 2023, pp. 967–976.

[5] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, and J. Ye, “INSIDE: LLMs' internal states retain the power of hallucination detection,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[6] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” arXiv:1610.01644, 2016.

[7] Y. Belinkov, “Probing classifiers: Promises, shortcomings, and advances,” Comput. Linguistics, vol. 48, no. 1, pp. 207–219, 2022.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in Advances in Neural Information Processing Systems 36, 2023.

[9] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd ed. Hillsdale, NJ, USA: Lawrence Erlbaum Associates, 1988.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[11] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. 2020 Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations, 2020, pp. 38–45.

[12] N. Carlini et al., “Extracting training data from large language models,” in Proc. 30th USENIX Security Symp., 2021, pp. 2633–2650.

[13] C. Burns, H. Ye, D. Klein, and J. Steinhardt, “Discovering latent knowledge in language models without supervision,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[14] H. Orgad, M. Toker, Z. Gekhman, R. Reichart, I. Szpektor, H. Kotek, and Y. Belinkov, “LLMs know more than they show: On the intrinsic representation of LLM hallucinations,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.

[15] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.

[16] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, “Detecting hallucinations in large language models using semantic entropy,” Nature, vol. 630, no. 8017, pp. 625–630, 2024.

[17] S. Kadavath et al., “Language models (mostly) know what they know,” arXiv:2207.05221, 2022.

[18] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in Text Summarization Branches Out, Barcelona, Spain, 2004, pp. 74–81.

[19] A. Vaswani et al., “Attention is all you need,” in Advances in Neural Information Processing Systems 30, 2017, pp. 5998–6008.